DevOrbit

Sprint 4 Acceptance Test Report

GitHub Integration Exit Checks (M1–M4)

User stories	US-31, US-32, US-33, US-34
Requirements	FR-11, FR-12, FR-35, NFR-4, NFR-11a, NFR-11b
Test type	Manual acceptance (black-box, end-to-end, real GitHub)
Tester	Urvish Babariya
Date	3 November 2026
Code version	`main` at `b234f22` (PR #29)
Repository	`https://github.com/Heer64/DevOrbit`

Contents

1 Purpose and scope **2**

2 Summary of results **2**

3 Test environment **3**
3.1 Setup . . . 3

4 Exit checks **4**
4.1 Check 1: Demo, connect GitHub and a repository . . . 4
4.2 Check 2: Real GitHub events appear in the Activity feed . . . 7
4.3 Check 3: Requests with a bad or missing signature are rejected . . . 8
4.4 Check 4: The app works while GitHub is unreachable . . . 9
4.5 Check 5: Disconnect, new events stop and old entries stay . . . 12
4.6 Bonus: Re-authorize after disconnect . . . 15

5 Observations and defects found **15**

6 Related automated tests **16**

7 Conclusion **17**

1 Purpose and scope

The Sprint 4 plan ends with exit checks for the GitHub integration that the automated tests cannot cover, because they need a real GitHub account, a real OAuth App and a real repository sending webhooks. This report records those checks, run by hand against the code on `main` after all eight Sprint 4 parts were merged.

- **M1, US-31** Authorize GitHub (OAuth, state check, token encrypted, never sent to the browser).
- **M2, US-32** Connect one repository; the connection shows as Active.
- **M3, US-33** Owner/Admin view status, disconnect, re-authorize; old feed entries stay.
- **M4, US-34** Signed webhooks for commits, issues and pull requests appear in the Activity feed.

The other Sprint 4 parts (M5 files, M6 docs, M7 dashboard, M8 completion and health) are covered by the automated backend, frontend and Playwright end-to-end tests and are not repeated here.

2 Summary of results

#	Exit check	Stories	Result
1	Demo: connect GitHub and a repository (Active, webhook active, ping received)	US-31, US-32, US-34	**PASS**
1b	A repository the account does not administer: connected, but no webhook and a clear message	US-32, US-34	**PASS**
2	A real commit and a real issue appear in the Activity feed with a GitHub link	US-34	**PASS**
3	A webhook request with a wrong or missing signature is rejected (401) and nothing is stored	US-34, NFR-4	**PASS**
4	The app keeps working while GitHub is unreachable; GitHub actions fail safely	NFR-11b	**PASS**
5	Disconnect: webhook removed, new events stop, old entries stay	US-33	**PASS**
B	Bonus: re-authorize after disconnect, connection Active again	US-33	**PASS**

3 Test environment

Machine	Windows laptop, Google Chrome, VS Code terminals (PowerShell)
Backend	`backend/`, `npm run dev` on `http://localhost:5000`
Frontend	`frontend/`, `npm run dev` on `http://localhost:5173`
Database	MongoDB Community Server 9.0.4 installed locally (`mongodb://127.0.0.1:27017/devorbit`)
GitHub OAuth App	"DevOrbit Local Test", callback `http://localhost:5000/api/github/callback`
Webhook relay	smee.io channel + `smee-client`, forwarding to `/api/github/webhook` (GitHub cannot reach `localhost`)
GitHub account	`@Urvish2007`
Test repository	`Urvish2007/devorbit-webhook-test` (public, owned by the tester, so the account is its admin)
Secrets	`GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`, `GITHUB_TOKEN_KEY` (64 hex), `GITHUB_WEBHOOK_SECRET`, `GITHUB_WEBHOOK_URL` set in `backend/.env` only; never committed and not shown in this report

3.1 Setup

1. Registered the OAuth App on GitHub with the homepage and callback URLs above (Figure 1).
2. Created a smee.io channel and started `smee-client` to forward deliveries to the local API.
3. Filled in the GitHub settings in `backend/.env`; the token key and webhook secret were generated with `crypto.randomBytes(32)`.
4. Before the settings were filled in, the server started normally and listed only the *names* of the missing settings, never any values (Figure 2). This matches US-31: secrets live only in server environment variables and are never logged.

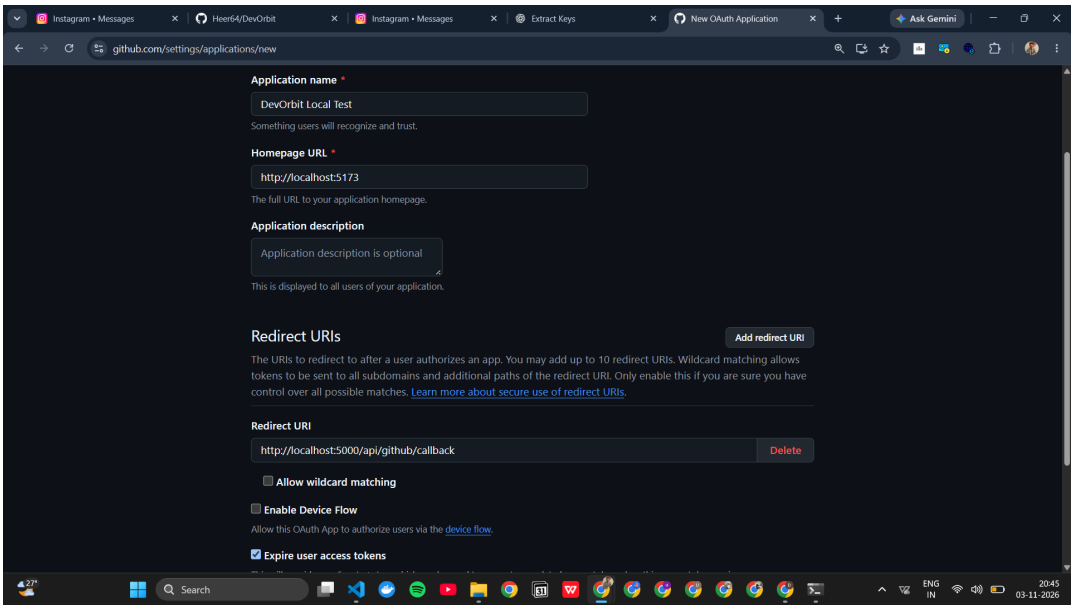


Figure 1: GitHub OAuth App registration: homepage `http://localhost:5173`, redirect URI `http://localhost:5000/api/github/callback`, wildcard matching and device flow off.

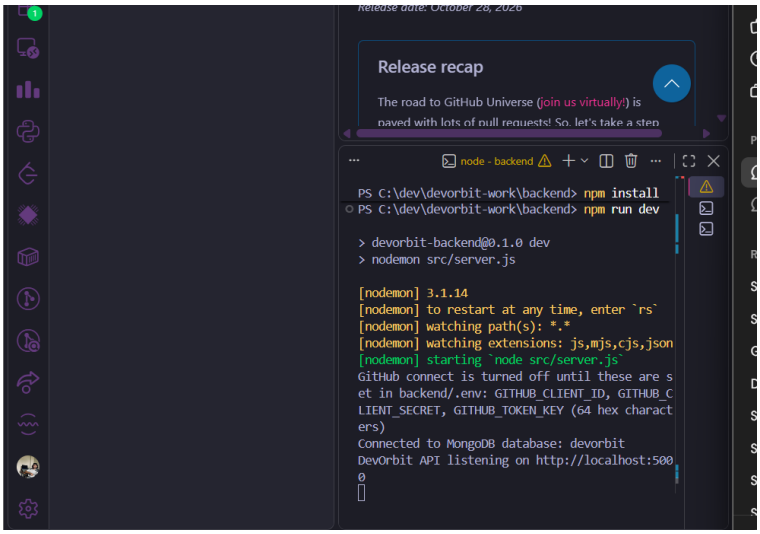


Figure 2: Backend before the GitHub settings were added: GitHub connect is turned off and only the setting names are printed. The API still starts and connects to MongoDB.

4 Exit checks

4.1 Check 1: Demo, connect GitHub and a repository

Objective	The Owner authorizes GitHub, picks one repository, the connection shows as Active and the webhook is created.
Requirement	US-31 (authorize with a state-checked callback), US-32 (one repository, shown as Active), US-34 (signed webhook configured).
Steps	Signed up, created a project, clicked *Connect GitHub*, approved on GitHub, clicked *Choose repository*, selected `Urvish2007/devorbit-webhook-test`, clicked *Connect repository*.
Expected	Panel shows *Active* and *Webhook active*; GitHub sends a `ping` that DevOrbit records.
Actual	Authorized as `@Urvish2007`; repository connected; status *Active*; badge *Webhook active*. smee.io received the `ping` from `Urvish2007/devorbit-webhook-test`, and *Recent deliveries* lists it as *Ignored* ("GitHub ping: the webhook reached DevOrbit"), which is the intended handling of a ping.
Result	**PASS**

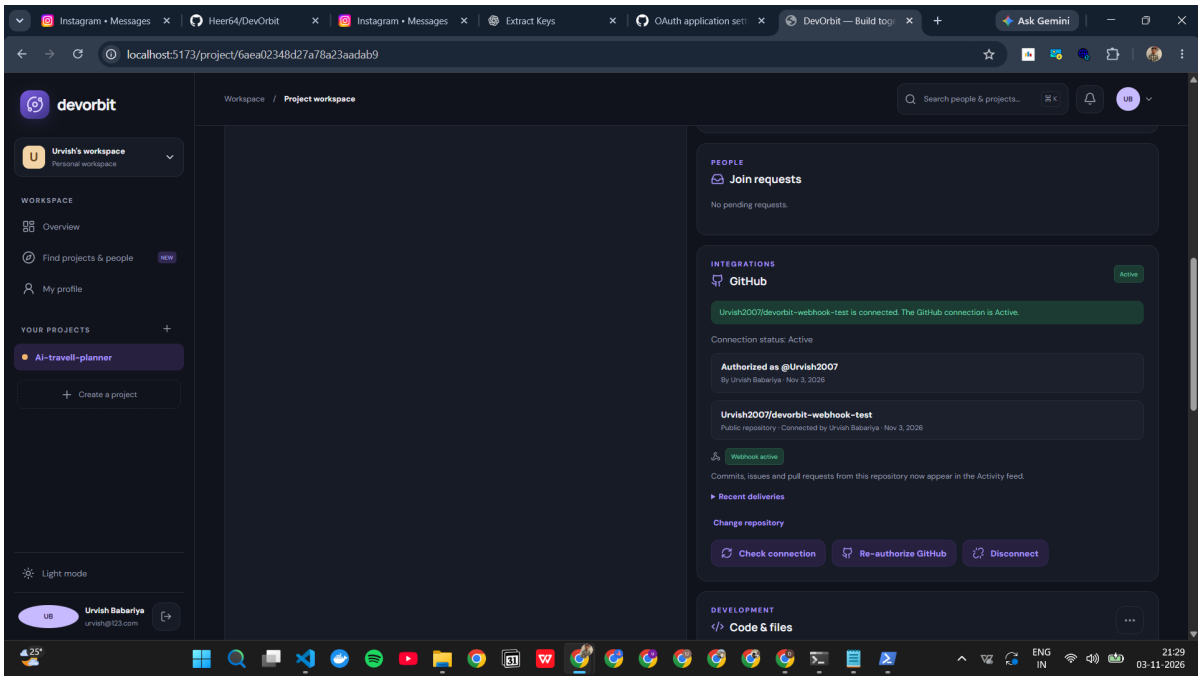


Figure 3: GitHub panel after connecting `Urvish2007/devorbit-webhook-test`: connection status Active, authorized as `@Urvish2007`, webhook active.

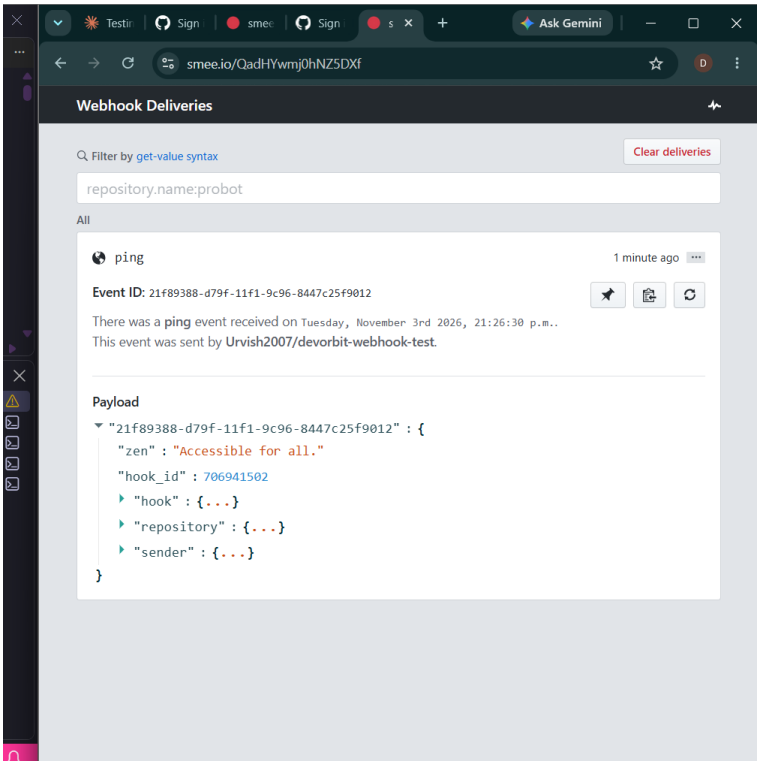


Figure 4: smee.io received GitHub's `ping` event from `Urvish2007/devorbit-webhook-test` when the webhook was created.

Check 1b: repository the account does not administer

Objective	GitHub only lets repository admins add webhooks. Connecting someone else's repository must not fail silently.
Steps	In the first attempt, the repository `NiranjanDO/WarpCopy-Parallel-File-Transfer-over-SSH` (owned by another account) was chosen.
Expected	The repository connects (US-32), but no webhook is created, and the panel explains why.
Actual	Status *Active*; badge *Webhook not set up* with the message "the authorized GitHub account is not an admin of this repository, so it cannot add a webhook". No deliveries. Switching with *Change repository* to the tester's own repository then worked (Check 1).
Result	**PASS**

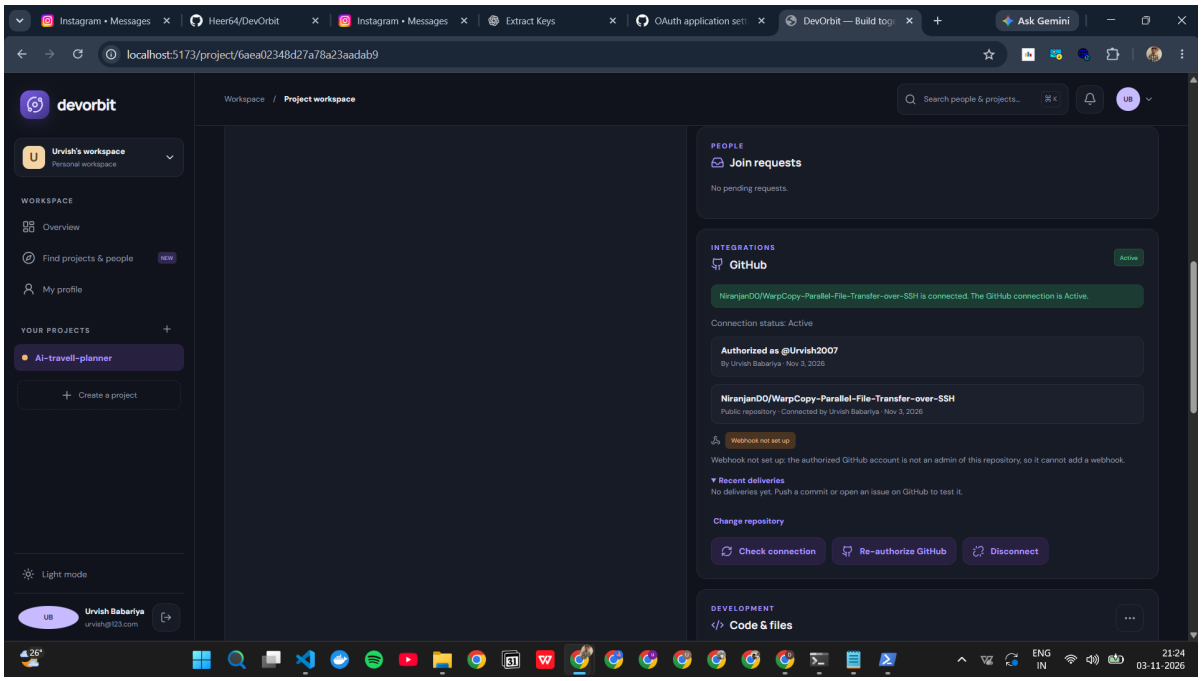


Figure 5: Non-admin repository: connected and Active, but the webhook is not set up and the reason is shown.

4.2 Check 2: Real GitHub events appear in the Activity feed

Objective	A commit pushed, or an issue opened, in the connected repository appears in the project's Activity feed with event type, title, author and a link to GitHub, within 60 seconds.
Requirement	US-34, FR-12.
Steps	On GitHub: edited `README.md` and committed to `main`; opened issue #1 "Test issue from exit check". In DevOrbit: opened the *Activity* tab and *Recent deliveries*.
Expected	Both events delivered (HTTP 202), processed, and shown in the feed with an *Open on GitHub* link.
Actual	smee forwarded three deliveries: `ping` (200) then `push` and `issues` (202 each). *Recent deliveries*: `push` and `issues` · `opened` both "Added to feed, 1 new entry". Activity feed: "Urvish2007 pushed commit 'Update README with webhook test information' to main" and "Urvish2007 opened issue #1 'Test issue from exit check'", each with *Open on GitHub*. Deliveries are answered at once and processed straight away, well inside the 60-second limit.
Result	**PASS**

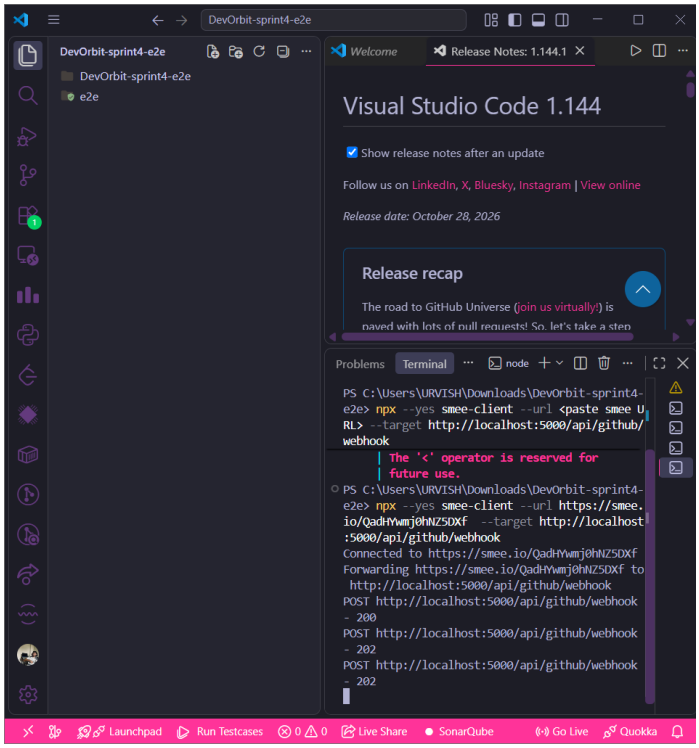


Figure 6: smee-client forwarding GitHub's deliveries to the local API: `ping` answered 200, `push` and `issues` accepted with 202.

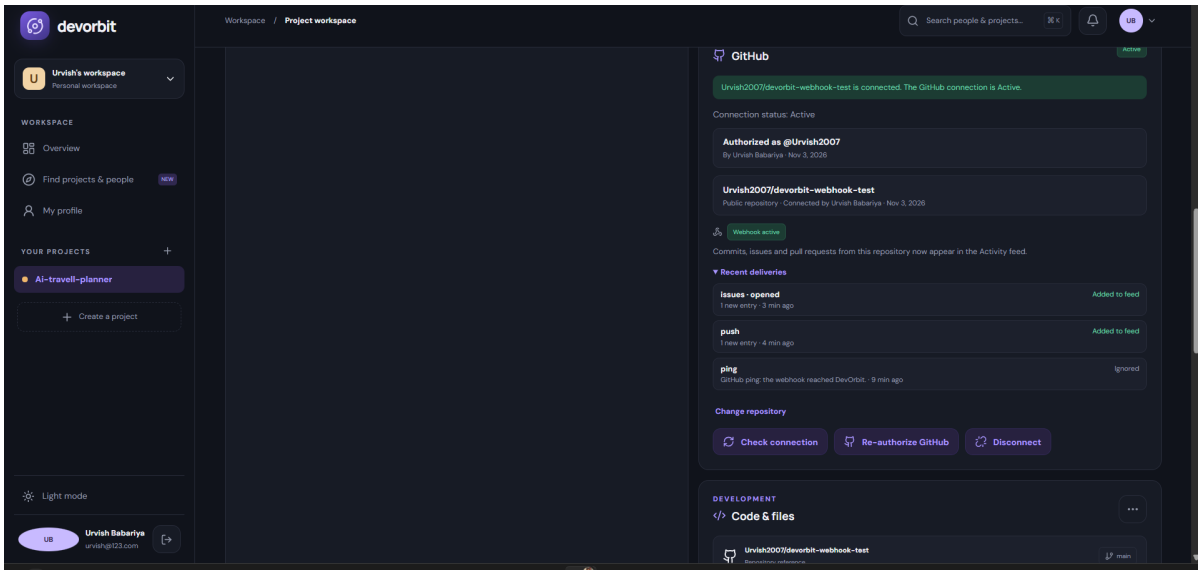


Figure 7: Recent deliveries in the GitHub panel: `issues · opened` and `push` added to the feed, `ping` ignored.

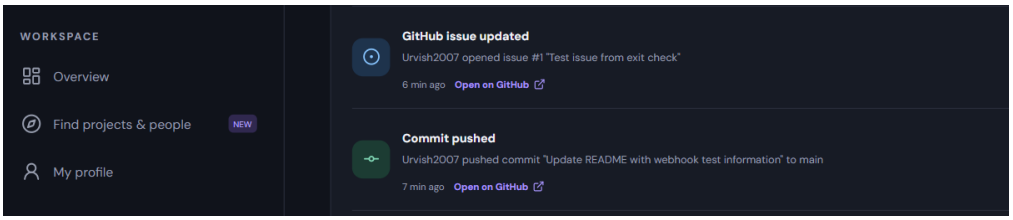


Figure 8: Activity feed entries created from GitHub, each with type, title, GitHub author and an *Open on GitHub* link.

4.3 Check 3: Requests with a bad or missing signature are rejected

Objective	Only deliveries signed with the shared secret are accepted (`X-Hub-Signature-256`, HMAC-SHA256 of the raw body).
Requirement	US-34 (verify the signature before processing), NFR-4.
Steps	Sent a fake `push` delivery with `curl.exe`: (a) with signature `sha256=000...0` (delivery `manual-test-1`); (b) with no signature header (delivery `manual-test-2`).
Expected	HTTP 401 for both; nothing stored in the feed or in Recent deliveries.
Actual	Both requests returned **401**. The Activity feed and Recent deliveries did not change; neither delivery ID was recorded.
Result	**PASS**

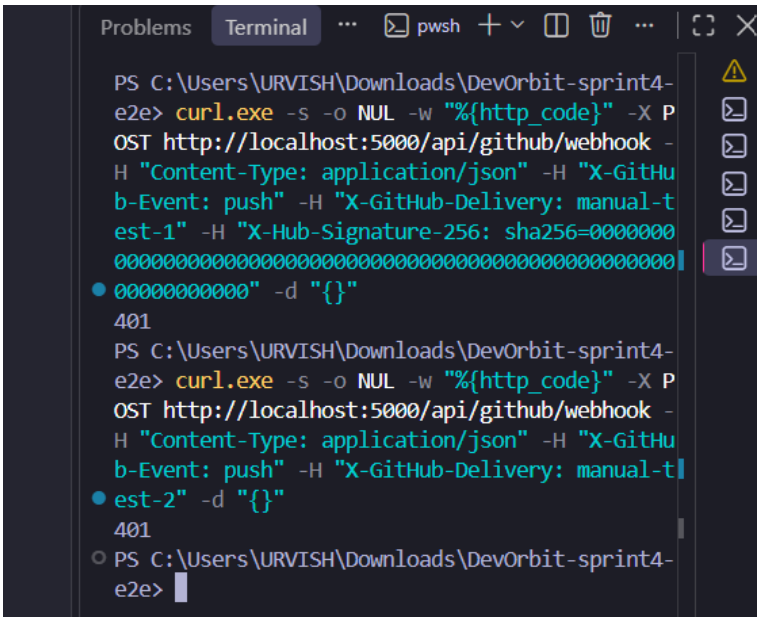


Figure 9: Fake deliveries rejected: wrong signature `sha256=000...0` returns 401, and a request without a signature returns 401.

4.4 Check 4: The app works while GitHub is unreachable

Objective	An outage at GitHub must not affect the rest of DevOrbit, and GitHub actions must fail without changing data.
Requirement	NFR-11b; Sprint 4 exit check "app works with GitHub down".
Steps	Blocked GitHub on the test PC by adding `127.0.0.1github.com` and `127.0.0.1api.github.com` to the Windows `hosts` file. Confirmed with `curl.exehttps://api.github.com` (result `000`, no connection). Then used the project: created a task, moved it on the board, sent a chat message, created a docs page, opened the Activity feed. Clicked *Check connection*. Finally removed the two lines and clicked *Check connection* again.
Expected	All project features work. *Check connection* shows an error and the status stays Active. After GitHub is reachable again, the check succeeds.
Actual	Task "Task while GitHub is down" created and moved To do → In progress; chat message "Chat works without GitHub" posted in `#general`; docs page "Offline note" created; all three logged in the Activity feed. *Check connection* showed "Could not reach GitHub. Please try again." and the status stayed *Active*, with the webhook still active. After unblocking: "Connection checked: the authorization works and `Urvish2007/devorbit-webhook-test` can be read."
Result	**PASS**

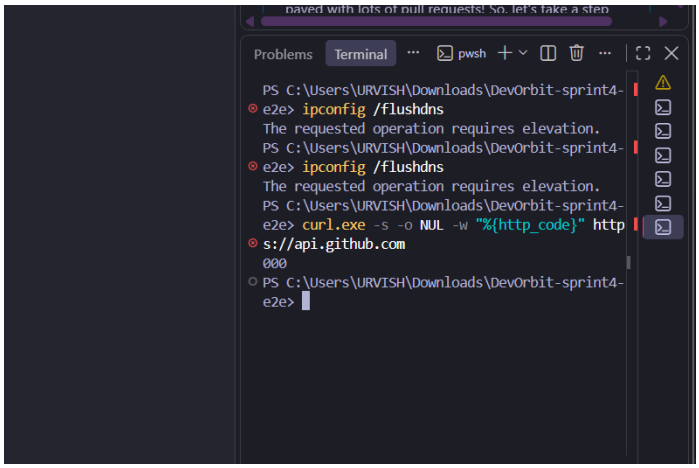


Figure 10: GitHub blocked: `curl.exe https://api.github.com` returns `000` (cannot connect).

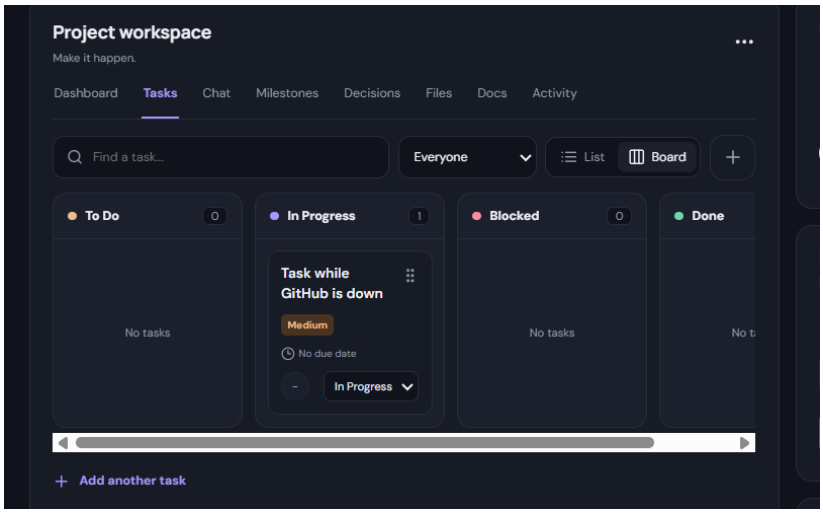


Figure 11: While GitHub is down: a new task created and moved to In Progress on the Kanban board.

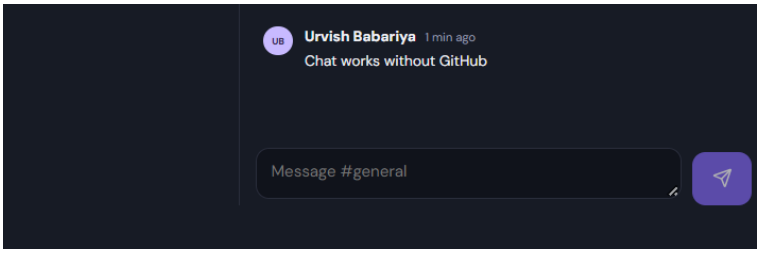


Figure 12: While GitHub is down: chat message posted in `#general`.

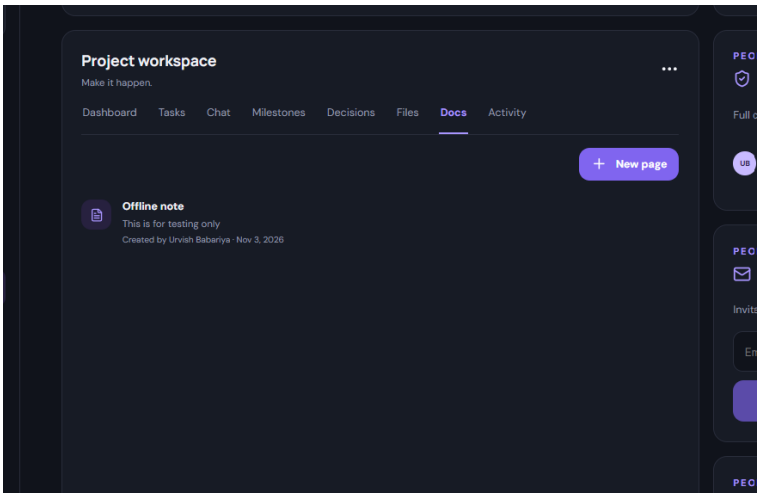


Figure 13: While GitHub is down: documentation page "Offline note" created.

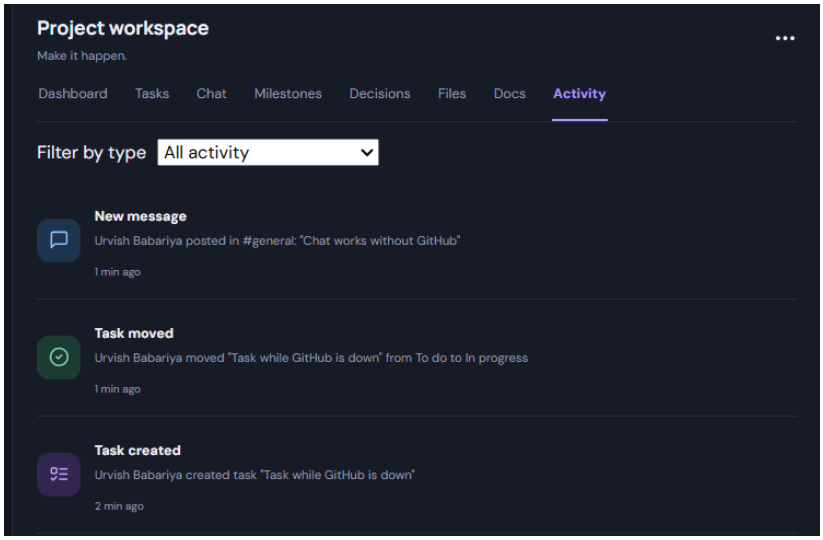


Figure 14: While GitHub is down: the Activity feed records the new task, the move and the chat message.

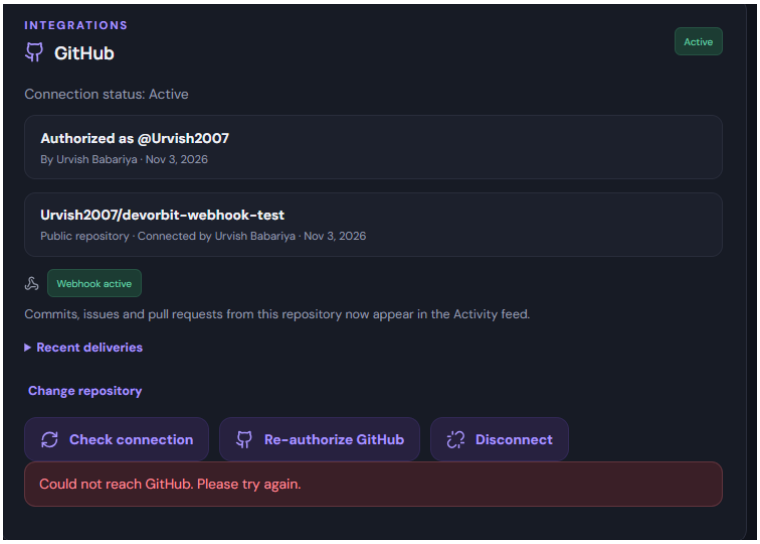


Figure 15: *Check connection* while GitHub is down: a clear error, and the connection stays Active with the webhook still active (nothing changed).

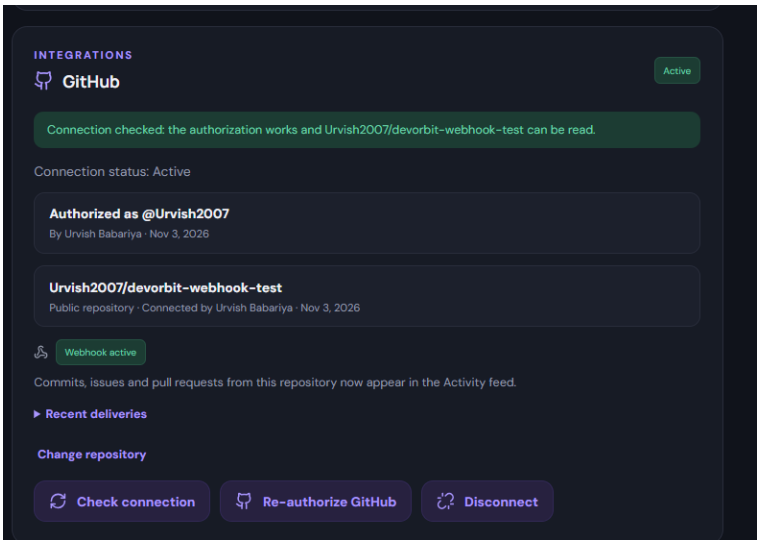


Figure 16: After GitHub was unblocked: *Check connection* succeeds.

4.5 Check 5: Disconnect, new events stop and old entries stay

Objective	After disconnect, no new GitHub events reach the feed, the webhook is removed from GitHub, and earlier feed entries remain.
Requirement	US-33, FR-35.
Steps	Clicked *Disconnect* and confirmed. While disconnected, on GitHub: committed the README line "After disconnect – should NOT appear" and opened issue #2 "Opened issue after Disconnect!". Then checked the Activity feed.
Expected	Status *Disconnected*; no feed entry for the commit or the issue made while disconnected; a "disconnected GitHub" entry in the feed; earlier entries are not deleted.
Actual	Status *Disconnected*: "GitHub was disconnected from this project. No new GitHub events are added to the Activity Feed. Earlier entries stay." The panel keeps the last account and repository and offers *Re-authorize GitHub*. Both events exist on GitHub (Figures 18 and 19), but the feed has **no entry for either**: directly after "Urvish Babariya disconnected GitHub (repository 'Urvish2007/devorbit-webhook-test')" the next entry is the re-authorization (Figure 20). The events from while it was disconnected were not added later either. Disconnect deletes no feed entries; the screenshot is cropped to the newest entries. The automated test `githubConnectionManagement.test.js` also checks that the webhook is removed from GitHub on disconnect; GitHub's Webhooks settings page was not captured in this run.
Result	**PASS**

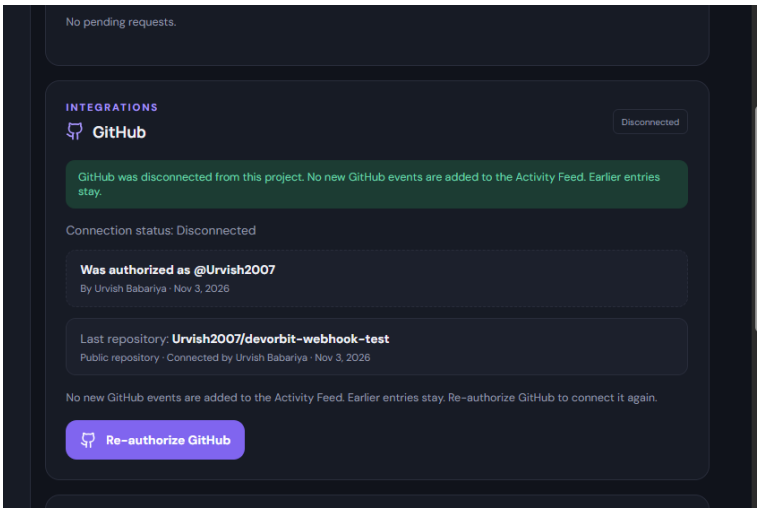


Figure 17: GitHub panel after Disconnect: status Disconnected, last account and repository kept, *Re-authorize GitHub* offered.

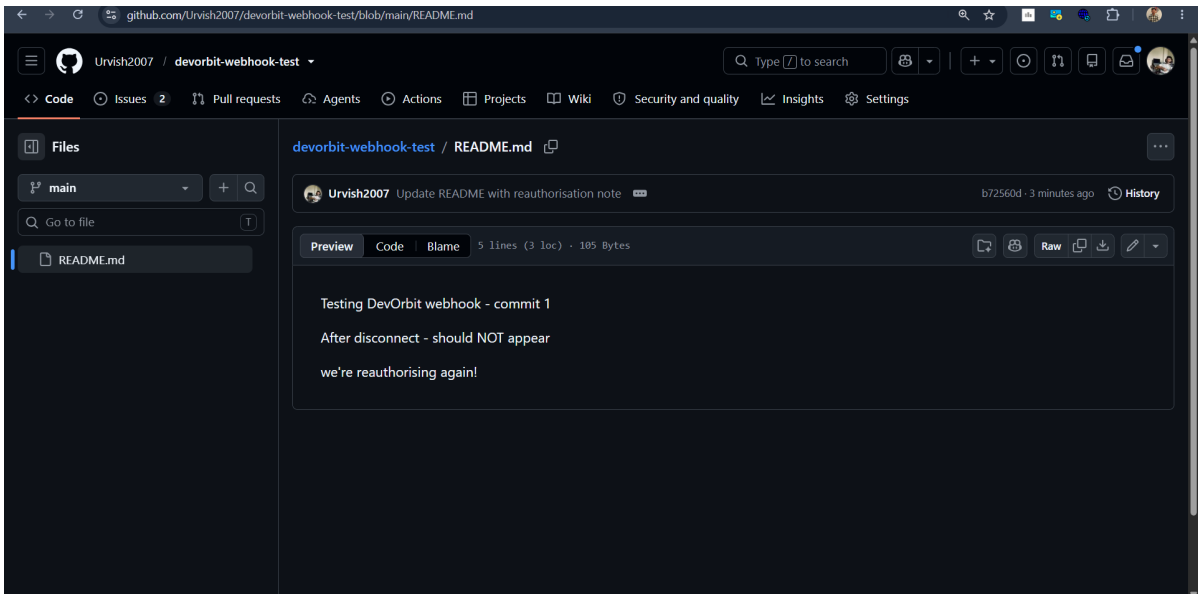


Figure 18: On GitHub, `README.md` contains the line "After disconnect – should NOT appear", committed while DevOrbit was disconnected (later lines were added after re-authorizing).

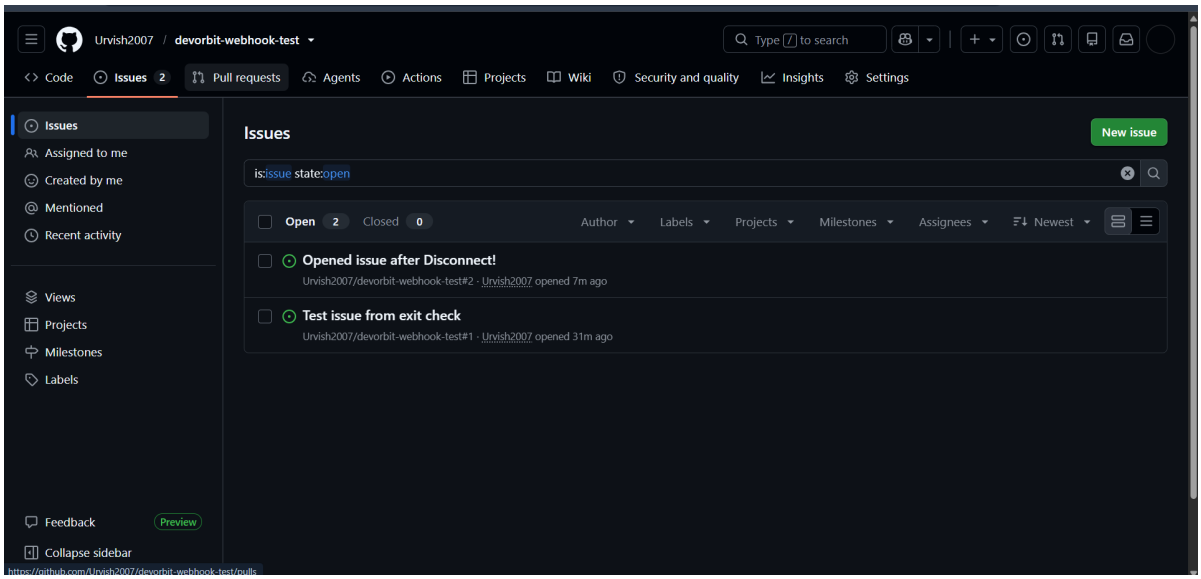


Figure 19: On GitHub, issue #2 "Opened issue after Disconnect!" was opened while DevOrbit was disconnected.

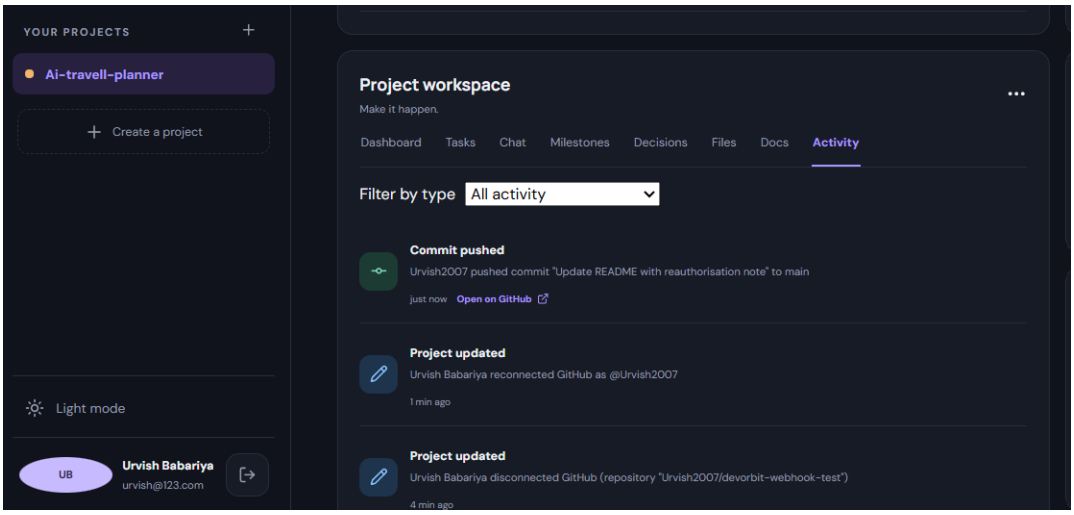


Figure 20: DevOrbit Activity feed, newest first: the commit after re-authorizing, "reconnected GitHub as @Urvish2007", and "disconnected GitHub". Nothing from the time it was disconnected (no README commit, no issue #2) was added.

4.6 Bonus: Re-authorize after disconnect

Objective	The Owner (or an Admin) can re-authorize a disconnected integration and it returns to Active with its repository.
Requirement	US-33.
Steps	Clicked *Re-authorize GitHub*, approved on GitHub, then committed "Update README with reauthorisation note" (README line "we're reauthorising again!").
Expected	The connection becomes Active again with the same repository, the re-authorization is logged, and new events reach the feed again.
Actual	The feed shows "Urvish Babariya reconnected GitHub as @Urvish2007", followed about a minute later by "Urvish2007 pushed commit 'Update README with reauthorisation note' to main" with *Open on GitHub* ("just now"). So the integration is back on the same repository and its webhook delivers again.
Result	**PASS**

See Figure 20: the "reconnected GitHub" entry and the commit that followed it.

5 Observations and defects found

ID	Severity	Observation	Recommendation
D1	Medium	When deliveries arrived through smee (which adds an `X-Forwarded-For` header), the backend logged `ERR_ERL_UNEXPECTED_X_FORWARDED_FOR` from `express-rate-limit` (Figure 21). Deliveries were still processed (202). The app does not set Express's `trust proxy`. On Render, which also sits behind a proxy, every client would then share the proxy's IP for rate limiting.	Set `app.set('trustproxy',1)` in `createApp()` for the deployed API (one proxy hop), ideally controlled by an environment variable, and re-test.
D2	Low	The Activity item for an issue that was *opened* is titled "GitHub issue updated" (its text correctly says "opened issue #1").	Title the item by action, e.g. "GitHub issue opened", or use a neutral "GitHub issue".
O1	Info	The OAuth App form had "Expire user access tokens" ticked. With it on, GitHub's token expires after 8 hours. The backend stores only the access token (no refresh), so the panel would then ask to re-authorize (handled by US-33).	For the team's OAuth App, untick it, or plan token refresh as future work.
O2	Info	On the test PC `ipconfig /flushdns` needs an administrator terminal (Figure 22). Windows applied the `hosts` change anyway (confirmed by `curl`, Figure in Check 4).	Note it in the test procedure; it does not affect the app.

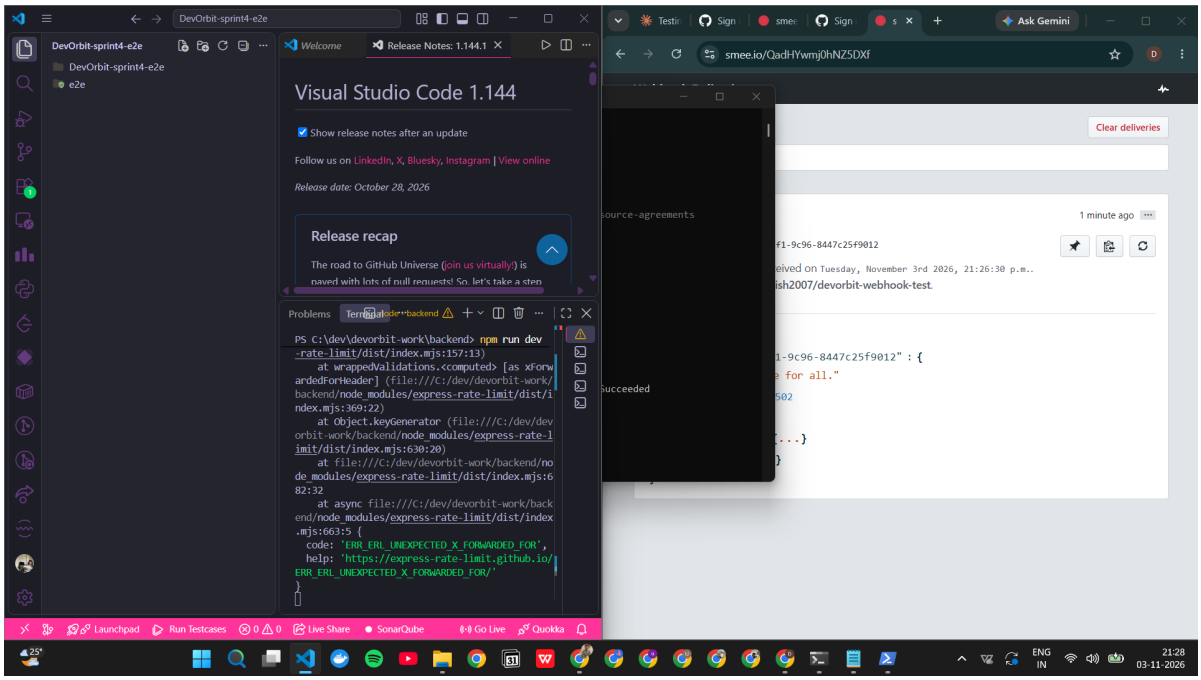


Figure 21: D1: `express-rate-limit` validation error logged while smee forwarded deliveries with an `X-Forwarded-For` header.

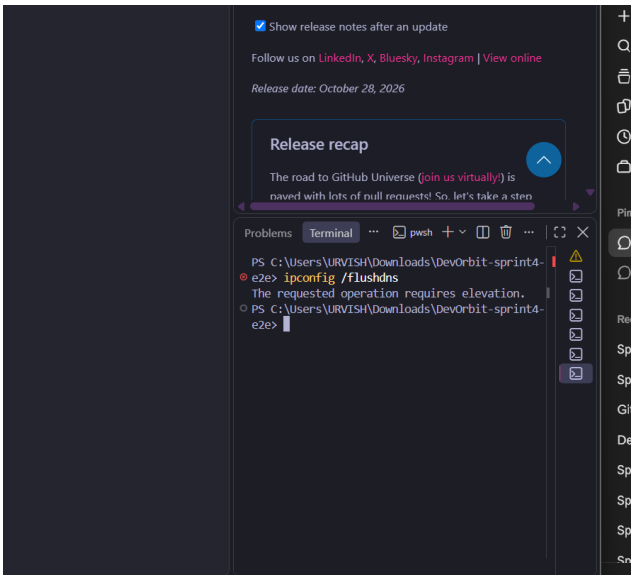


Figure 22: O2: `ipconfig /flushdns` requires an elevated (administrator) terminal on the test PC.

6 Related automated tests

The same rules are also covered by automated tests on `main`, which run in GitHub Actions on every pull request (Node 20 and 22, MongoDB 7). They use a fake GitHub, so this manual run is what proves the real integration.

Part	Backend (node:test + supertest)	Frontend (Vitest + Testing Library)
M1 US-31	`githubOAuth.test.js`, `githubCrypto.test.js`	`GithubIntegrationPanel.test.jsx`
M2 US-32	`githubRepos.test.js`, `githubRepository.test.js`	`GithubRepository.test.jsx`
M3 US-33	`githubConnection.test.js`, `githubConnectionManagement.test.js`	`GithubConnectionManagement.test.jsx`
M4 US-34	`githubEvents.test.js`, `githubWebhook.test.js`, `githubWebhooks.test.js`	`GithubEvents.test.jsx`

Examples of what they check: a missing or wrong `X-Hub-Signature-256` returns 401 and stores nothing; a repeated delivery is not processed twice; after disconnect, new events stop and old entries stay; disconnect and *Check connection* behave safely when GitHub is down or rate-limited; the token never appears in a response or a log line.

7 Conclusion

Checks 1–4 passed against real GitHub: authorization, repository connection, webhook creation, delivery of real commit and issue events into the Activity feed, rejection of unsigned or wrongly signed requests, and correct behaviour while GitHub is unreachable. Disconnect stopped new events (a commit and an issue made while disconnected never reached the feed), and re-authorizing restored the connection on the same repository, with new commits arriving again. All Sprint 4 GitHub exit checks passed. Defect D1 (`trust proxy`) should be fixed before the API is relied on behind Render's proxy; D2 is a wording issue.

Tested by: _____
Urvish Babariya (tester)

Date: 3 November 2026

Reviewed by: _____

Date: _____